Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la ingeniería

Ingeniería

Estructura de Datos Sección “A”

Ing. Mario Moises Ramirez Tobar

Primer Semestre 2026

Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

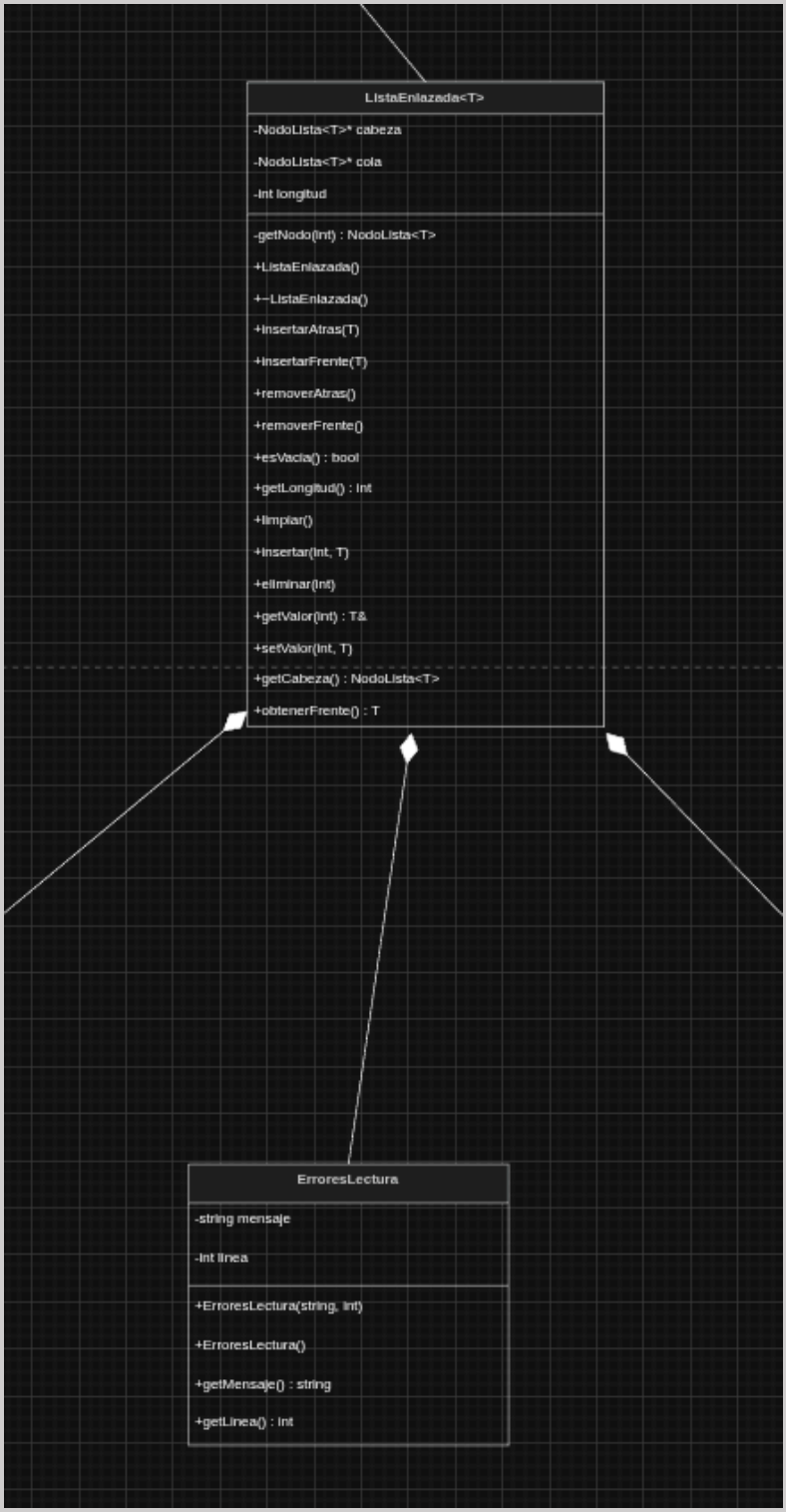
Big-O y justificacion Proyecto #1

Descripción:

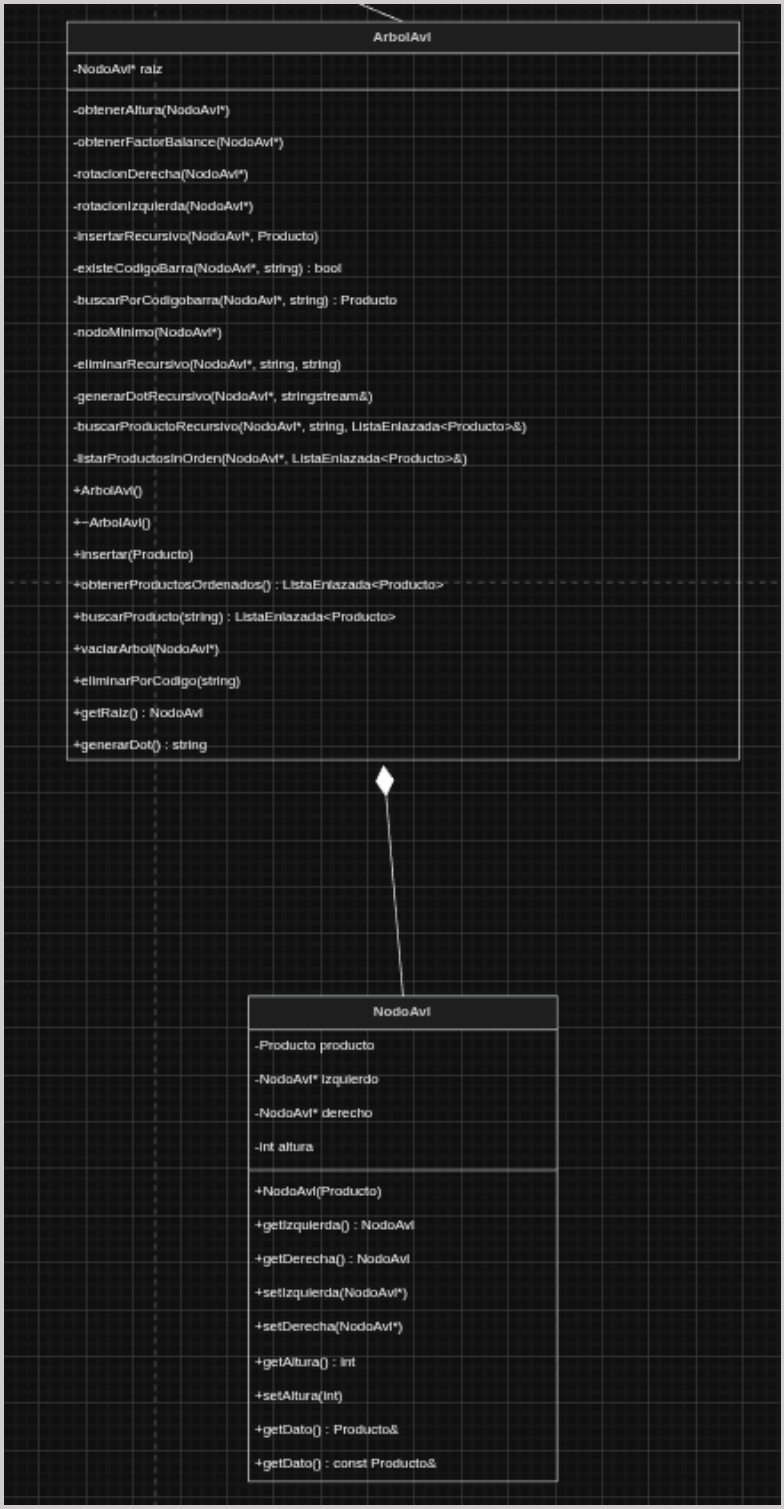
En el siguiente documento se le dará a conocer el análisis profundo de cómo es que se basan las estructuras de datos para poder generar las funcionalidades de almacenar productos dentro del proyecto. Sobre todo se tiene la justificación del porqué se tomaron ciertas decisiones al momento de diseñar los árboles y qué caminos siguen para poder organizar los datos tal como se presentan en el respectivo proyecto. **La llave primaria elegida para poder dar integridad a los datos fue el código de barra de los productos.**

Diseño de las estructuras

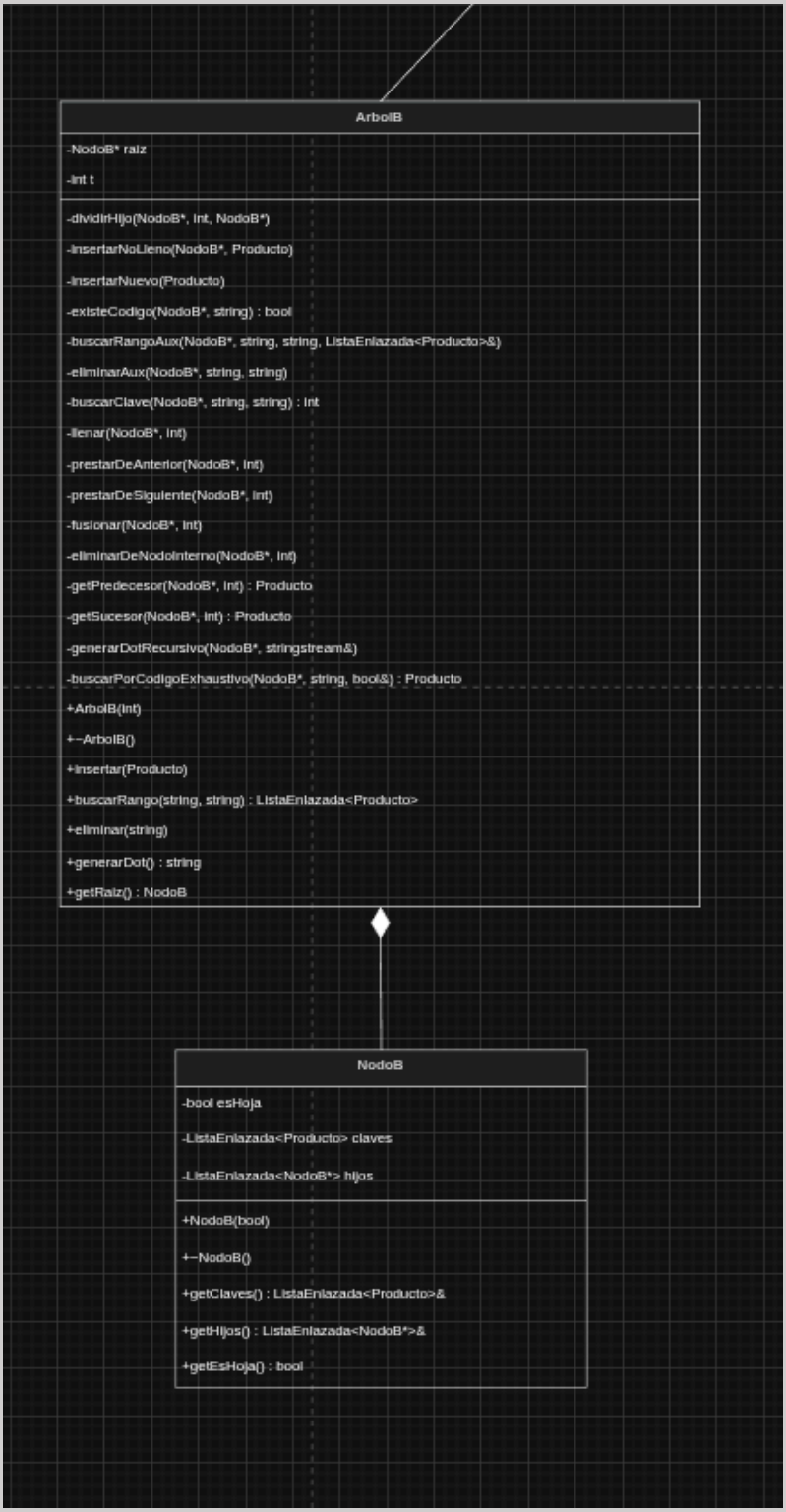
Lista enlazada:



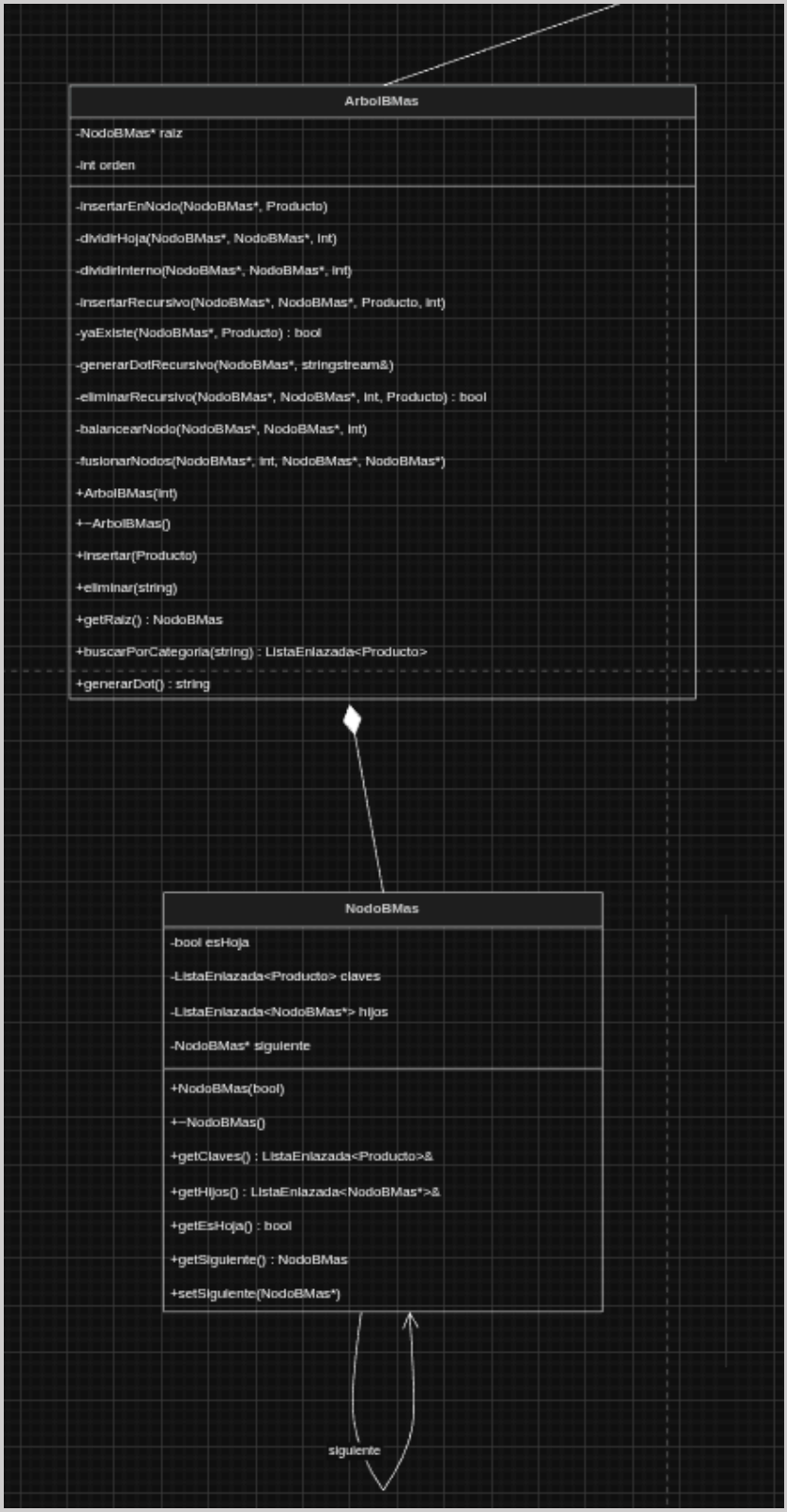
Arbol AVL:


ArbolAvl
-NodoAvl* raiz
-obtenerAltura(NodoAvl*)
-obtenerFactorBalance(NodoAvl*)
-rotacionDerecha(NodoAvl*)
-rotacionIzquierda(NodoAvl*)
-insertarRecursivo(NodoAvl*, Producto)
-existeCodigoBarra(NodoAvl*, string) : bool
-buscarPorCodigobarra(NodoAvl*, string) : Producto
-nodoMinimo(NodoAvl*)
-eliminarRecursivo(NodoAvl*, string, string)
-generarDotRecursivo(NodoAvl*, stringstream&)
-buscarProductoRecursivo(NodoAvl*, string, ListaEnlazada<Producto>&)
-listarProductosInOrden(NodoAvl*, ListaEnlazada<Producto>&)
+ArbolAvl()
+~ArbolAvl()
+insertar(Producto)
+obtenerProductosOrdenados() : ListaEnlazada<Producto>
+buscarProducto(string) : ListaEnlazada<Producto>
+vaciarArbol(NodoAvl*)
+eliminarPorCodigo(string)
+getRaiz() : NodoAvl
+generarDot() : string
NodoAvl
-Producto producto
-NodoAvl* Izquierdo
-NodoAvl* derecho
-int altura
+NodoAvl(Producto)
+getIzquierda() : NodoAvl
+getDerecha() : NodoAvl
+setIzquierda(NodoAvl*)
+setDerecha(NodoAvl*)
+getAltura() : int
+setAltura(int)
+getDato() : Producto&
+getDato() : const Producto&

Arbol B:


ArbolB
-NodoB* raiz
-int t
-dividirHijo(NodoB*, int, NodoB*)
-insertarNoLleno(NodoB*, Producto)
-insertarNuevo(Producto)
-existeCodigo(NodoB*, string) : bool
-buscarRangoAux(NodoB*, string, string, ListaEnlazada<Producto>&)
-eliminarAux(NodoB*, string, string)
-buscarClave(NodoB*, string, string) : int
-llenar(NodoB*, int)
-prestarDeAnterior(NodoB*, int)
-prestarDeSiguiente(NodoB*, int)
-fusionar(NodoB*, int)
-eliminarDeNodoInterno(NodoB*, int)
-getPredecesor(NodoB*, int) : Producto
-getSucesor(NodoB*, int) : Producto
-generarDotRecursivo(NodoB*, stringstream&)
-buscarPorCodigoExhaustivo(NodoB*, string, bool&) : Producto
+ArbolB(int)
+~ArbolB()
+insertar(Producto)
+buscarRango(string, string) : ListaEnlazada<Producto>
+eliminar(string)
+generarDot() : string
+getRaiz() : NodoB
NodoB
-bool esHoja
-ListaEnlazada<Producto> claves
-ListaEnlazada<NodoB*> hijos
+NodoB(bool)
+~NodoB()
+getClaves() : ListaEnlazada<Producto>&
+getHijos() : ListaEnlazada<NodoB*>&
+getEsHoja() : bool

Arbol B+:


ArbolBMas
-NodoBMas* raiz
-int orden
-insertarEnNodo(NodoBMas*, Producto)
-dividirHoja(NodoBMas*, NodoBMas*, int)
-dividirInterno(NodoBMas*, NodoBMas*, int)
-insertarRecursivo(NodoBMas*, NodoBMas*, Producto, int)
-yaExiste(NodoBMas*, Producto) : bool
-generarDotRecursivo(NodoBMas*, stringstream&)
-eliminarRecursivo(NodoBMas*, NodoBMas*, int, Producto) : bool
-balancearNodo(NodoBMas*, NodoBMas*, int)
-fusionarNodos(NodoBMas*, int, NodoBMas*, NodoBMas*)
+ArbolBMas(int)
+~ArbolBMas()
+insertar(Producto)
+eliminar(string)
+getRaiz() : NodoBMas
+buscarPorCategoria(string) : ListaEnlazada<Producto>
+generarDot() : string
NodoBMas
-bool esHoja
-ListaEnlazada<Producto> claves
-ListaEnlazada<NodoBMas*> hijos
-NodoBMas* siguiente
+NodoBMas(bool)
+~NodoBMas()
+getClaves() : ListaEnlazada<Producto>&
+getHijos() : ListaEnlazada<NodoBMas*>&
+getEsHoja() : bool
+getSiguiente() : NodoBMas
+setSiguiente(NodoBMas*)
siguiente

Análisis de complejidad Lista Enlazada No Ordenada

En este proyecto se utilizó una lista doblemente enlazada. Tal como fue vista en clase esta al tener un doble enlace es consciente de su nodo anterior y del siguiente. Por lo tanto permite insertar objetos o eliminar objetos en cualquier posición ya que cada **Nodo** apunta hacia su nodo anterior y siguiente.

Donde cada nodo contiene:

- Un dato genérico **T**
- Un puntero al siguiente nodo
- Un puntero al nodo anterior

Su complejidad pese a que esta pueda tener **n** número de elementos su complejidad no depende de su tamaño por lo tanto se dice que:

Inserción: Este es el proceso en donde se registra un nuevo producto dentro de esta. Y al estar desordenada no se preocupa por ordenar cada vez que inserta. Por lo tanto su complejidad es:

- Crear nodo -> O(1)
- Ajustar punteros -> O(1)
- Actualizar head -> O(1)
- Guardar nodo -> O(1)

Búsqueda: Al generar una búsqueda de cierto nodo en un índice, su complejidad puede variar ya que se tiene que recorrer nodo por nodo pero solo es en el peor de los casos cuando son demasiados datos el tiempo puede variar. Pero esta en promedio tiene una complejidad de:

O(n)

Eliminación: Esta aplica el mismo factor al momento de eliminar ya que hay que buscar el nodo y ajustar el puntero. Entonces se puede decir que también su complejidad viene siendo prácticamente similar a la de una inserción con el factor de búsqueda. Esto tomando en cuenta el peor de los casos también cuando una lista desordenada puede tener la casualidad o no de encontrar rápido el producto:

O(n)

Cálculos de tiempos empíricos:

En este proyecto fue muy importante cronometrar cuánto tiempo le tomaba a cada lista buscar datos. En este caso está siempre insertaba atrás, de tal forma que se comportaba en forma de cola.

Promedio de comparaciones respecto a los 1000 productos evaluados como pruebas:

n/2 = 500 comparaciones

Por lo tanto se puede conjeturar que el costo de búsquedas toma la misma complejidad de la lista pero variando en tiempos dependiendo que tan lejos esté el dato.

Ejemplo empírico: (Casos muy extremos)

- Búsquedas:
    - Por categoría: 1.7 - 2.0 ms
    - Por fecha de expiración: 1.7 - 3.3 ms
    - Por Nombre: 3.1 - 3.3 ms
- 5 veces

Promedio:

- Por categoría: 1.85 - 2.0 milisegundos aproximadamente en adelante
- Por fecha de expiración: 2.5 - 2.9 milisegundos aproximadamente en adelante
- Por Nombre: 3.20 - 3.25 milisegundos aproximadamente en adelante

Justificación:

Estos resultados se obtienen y se dan un poco desde el punto de vista de pruebas humanas. Y mi argumento sobre los tiempos obtenidos se debe a que da la casualidad de que esta lista no ordenada en ciertos panoramas es más eficiente y en otros no. Ya que por su mismo factor de aleatoriedad de búsqueda por parte del usuario esta puede tardar más o menos tiempo. Sin embargo su promedio sí que es significante ya que tarda en torno a más de 2 segundos en promedio de los tiempos. Estos segundos son vitales para un humano ya que el simple hecho de tardar segundos la hace lenta. Y sus razones tiene ya que como tal su búsqueda es secuencial y no tiene la capacidad de buscar de otras formas más que recorrer nodo a nodo la cantidad de veces que haga falta hasta buscar qué valor es igual o no. Como depende de este mismo factor la suerte es mucho más rápida incluso en relación a la que esta ordenada.

Análisis de complejidad Lista Ordenada

En el caso de la lista ordenada era otra estructura que se comparaba con las demás, con la característica que siempre se mantiene ordenada. Esto para las búsquedas era demasiado óptimo. Sin embargo para la inserción era más tardada ya que tenía que ir comparando donde insertarse y después ordenarse para siempre estar de la forma que se esperaba.

Donde cada nodo contiene:

- Un dato genérico **T**
- Un puntero al siguiente nodo
- Un puntero al nodo anterior

Su complejidad pese a que esta pueda tener **un número** de elementos su complejidad no depende de su tamaño. Incluso llega a ser favorable y más eficiente que se mantenga ordenada. O en casos extremos si pierde rendimiento por lo tanto se dice que:

Inserción: Este es el proceso en donde se registra un nuevo producto dentro de esta. Y al estar ordenada es un poco más ágil en ciertos panorama sin embargo esto no aumenta como tal su complejidad. Solo es importante destacar que pese a mantener la misma complejidad de inserción:

- Crear nodo -> O(1)
- Ajustar punteros -> O(1)
- Actualizar head -> O(1)
- Guardar nodo -> O(1)

Esta Lista como se construye ordenada en cualquier panorama. Llega a tener una complejidad muy costosa porque tiene que insertarse y luego ordenarse dando una complejidad de:

$$\mathbf{O}(n^2)$$

Búsqueda: Al generar una búsqueda de cierto nodo en un índice, su complejidad puede variar ya que se tiene que recorrer nodo por nodo pero solo es en el peor de los casos cuando son demasiados datos puede tardar más tiempo pero siempre tiene una complejidad de:

O(n)

En este caso también es favorable la eficiencia de búsqueda ya que media vez encuentra corta el flujo y siempre seguirá cierta secuencia. Siendo un poco más rápida que la no ordenada.

Eliminación: Esta aplica el mismo factor al momento de eliminar ya que hay que buscar el nodo y ajustar el puntero. Entonces se puede decir que también su complejidad se mantiene igual con el estándar de lista enlazada. Esto tomando en cuenta el peor de los casos también cuando una lista ordenada puede tener la casualidad o no de encontrar rápido el producto dando como resultante:

O(n)

Cálculos de tiempos empíricos:

En este proyecto fue muy importante cronometrar cuánto tiempo le tomaba a cada lista buscar datos. En este caso está siempre insertaba y luego ordenaba, de tal forma que le permite generar búsquedas más rápidas sin embargo con el factor de que si el producto quedaba muy lejos le llevaría más tiempo.

Promedio de comparaciones respecto a los 1000 productos evaluados como pruebas:

n/2 = 500 comparaciones

Por lo tanto se puede conjeturar que el costo de búsquedas toma la misma complejidad de la lista pero variando en tiempos dependiendo que tan lejos esté el dato.

Ejemplo empírico: (Casos muy extremos)

- Búsquedas:

    - Por categoría: 0.8 - 1.2 ms
    - Por fecha de expiración: 3.0 - 3.5 ms
    - Por Nombre: 3.0 - 3.5 ms
- 5 veces

Promedio:

- Por categoría: 1.9 - 2.2 milisegundos aproximadamente en adelante
- Por fecha de expiración: 3.25 - 3.6 milisegundos aproximadamente en adelante
- Por Nombre: 3.25-3.5 milisegundos aproximadamente en adelante

Justificación:

Estos resultados se obtienen y se dan un poco desde el punto de vista de pruebas humanas. Y mi argumento sobre los tiempos obtenidos se debe a que da la casualidad de que está lista ordenada en algunos panoramas también corre con la suerte de ser más rápida cuando realmente se sigue un orden de peticiones. Permitiendo retornar datos más rápidos, pero por lo mismo de estar ordenada tarda muchísimo más que la desordenada ya que no encuentra datos por casualidad sino que tiene un criterio propio. Pero esto no la hace superior incluso me atrevo a decir que la hace más ineficiente que la que no está ordenada ya que en casos extremos como lo es el orden alfabético es pésima porque tiene que buscar extremos. Y esto para el punto de vista humano es increíblemente malo. Incluso experimentando me tope con tiempos hasta de 10 segundos buscando. Así que no es del todo fiable para un sistema enorme.

Análisis de complejidad Árbol AVL

Esta es una de las estructuras especiales del programa ya que involucra a la especialización de los árboles de búsqueda. Conocido como el árbol AVL que es un árbol binario balanceado que por su búsqueda tan eficiente permite tiempos increíblemente cortos en búsqueda de datos.

Búsqueda: El árbol AVL posee la característica de ejercer una búsqueda binaria y este nombre se le atribuye ya que siempre decide entre dos caminos que tomar. Ya que a cada paso que da descarta al 50% de datos restantes. Esta reducción es increíble ya que se obtienen resultados muy rápido. Definiendo una complejidad de búsqueda es sublineal, representada como:

O(log(n))

Inserción: Ésta consta de dos fases. Mezclando tanto la búsqueda y el rebalanceo de los nodos. La búsqueda tiene la complejidad ya mencionada y al subir por la recursión se actualizan las alturas y se hacen rotaciones dependiendo el caso. Que es una reacomodación de punteros constante **O(1). Estas se harán como máximo una serie hasta llegar a la raíz.**

Dando una complejidad de: O(1) + O(log(n))

O(log(n))

Eliminación: Esta tiene una complejidad similar a la de la inserción ya que se busca el nodo para poderlo eliminar y posteriormente se procede a realizar las rotaciones respectivas acorde a como se desequilibre el árbol.

Dando una complejidad de: O(1) + O(log(n))

O(log(n))

Nota: Se declararon estas complejidades debido a que gracias a la recursión se aprovecha mucho el stack de llamadas para poder balancear los nodos.

Recorrido in-order: Esta es la forma de listar en orden alfabético todos los nombres que se ingresan en el árbol. Ya que si el árbol AVL está regido por los nombres de los productos y balancearse acorde a ese parámetro enviado por un mismo camino las duplicidades. Permite listar fácilmente recorriendo el árbol y simplemente agregando todos los datos a una lista que posteriormente se imprimirá.

- Lado Izquierdo
- Raiz
- Lado Derecho

Cálculos de tiempos empíricos:

En este proyecto fue muy importante cronometrar cuánto tiempo le tomaba al árbol AVL buscar datos. En este caso está siempre insertaba con la garantía que siempre estaría ordenado, debido a que este es un árbol binario solo existían 2 caminos facilitando demasiado la tarea de búsqueda.

Promedio de comparaciones respecto a los 1000 productos evaluados como pruebas:

log(n) -> log(1000) = 10 comparaciones

Ejemplo empírico: (Casos muy extremos)

- Búsquedas:
  - Por Nombre: 0.011 - 0.015 ms
- 5 veces

Promedio:

- Por puntual por Nombre: 0.011 - 0.0150 milisegundos aproximadamente en adelante
- Por extremos: 0.0095-0.010 milisegundos aproximadamente en adelante

Justificación: El árbol AVL es el mejor en su rama de comparación en contra las listas ya que este por su búsqueda binaria descarta por cada comparación prácticamente el 50% de los datos. Llegando de esta forma muy rápida al resultado de búsqueda ya que todo gracias a la recursión se puede obtener de forma eficiente. En el caso de la implementación realizada en el proyecto se tomó el camino de siempre enviar por el camino derecho los productos con nombres iguales para delegar los roles de las rotaciones a los nodos y de esta forma al buscar inorden se pueden localizar muy rápido y cortar el flujo media vez se termine de propagar la rama. Teniendo una velocidad de aproximadamente 250 veces mayor a la lista enlazada incluso. Se la lleva muy por delante este árbol.

Análisis de complejidad Árbol B

Esta es la segunda estructura de datos más potente que tiene el programa teniendo un árbol B. Este árbol por decisión del programador se declaró de grado 5 y su parámetro de orden la fecha de expiración. Por lo tanto se tenían:

- 5 claves y como máximo 9 antes del desborde
- Por lo tanto para la realización del proyecto se tenían listas de máximo 10 hijos. Esto con el fin de poder manejar el árbol de la manera correcta respetando las 9 claves disponibles.

Este es uno de los árboles más increíbles ya que gracias al nivel que se le dio apenas si escala verticalmente. Esta decisión fue tomada debido a que los volúmenes de datos esperados para este proyecto y brindar escalabilidad horizontal de que escale verticalmente de forma más amigable. Se calcula que para 10000 datos aproximadamente se tendría una altura de:

$$\log(10000) = 4 \text{ niveles}$$

Búsqueda: Este árbol se basa en comparar en su lista de claves que tiene 2d -1 claves y teniendo 2d nodos esto para poder generar las conexiones en base al movimiento de los índices por ciclos. buscando el intervalo en el que hacen match las fechas. Para este árbol se tiene una complejidad de búsqueda del:

$$\mathbf{O}(log_m(\mathbf{n}))$$

Inserción: Esta va de la mano también con la búsqueda sin embargo tiene que ir verificando recursivamente si se desborda la lista de claves se propaga la separación de la lista hacia la raíz para poder ir verificando de tal forma de saber si se desencadena la partición de las listas. Sin embargo esto sigue siendo con la misma complejidad linealitmica:

$$\mathbf{O}(log_m(\mathbf{n}))$$

Nota: El parámetro de inserción debido a que pueden haber fechas iguales se concatenan la llave primaria que era el código de barras con la fecha para poder seguir ordenando alfabéticamente.

Inserción: Esta va de la mano también con la búsqueda sin embargo tiene que ir verificando recursivamente si se desborda la lista de claves se propaga el balanceo mediante los préstamos y fusiones que pueden existir de la lista de claves hacia la raíz para poder ir verificando de tal forma de saber si se desencadenan las rotaciones o fusiones que se deben de hacer cuando este árbol elimina datos. Sin embargo esto sigue siendo con la misma complejidad linealitmica:

$$\mathbf{O}(log_m(\mathbf{n}))$$

Cálculos de tiempos empíricos:

En este proyecto fue muy importante cronometrar cuánto tiempo le tomaba al árbol B buscar datos. En este caso está siempre insertaba con la garantía que siempre estaría ordenado por el parámetro de orden del árbol que era la fecha. El promedio de comparaciones respecto a los 1000 productos evaluados como pruebas:

$$log_{10}(\mathbf{n}) \text{ -> } log_{10}(\mathbf{1000}) = \mathbf{3\ saltos}$$

Ejemplo empírico: (Casos muy extremos)

- Búsquedas:
    - Por fecha: 0.020 - 0.050 ms
- 5 veces

Promedio:

- Búsqueda puntual por rangos de fecha: 0.035 - 0.060 milisegundos aproximadamente en adelante
- Por extremos: 0.020-0.030 milisegundos aproximadamente en adelante

Justificación: El uso de listas enlazadas para poder crear el árbol B fue tomada debido a que en su debido momento se pensó a una escala de miles de datos. Sin embargo es importante tomar en cuenta que esto puede afectar al rendimiento si se colocan millones de datos, pero esto también tiene un trasfondo y es que generar la copia de arreglos al momento en el que se desbordan las claves está causaba problemas al perder referencias sin embargo es importante resaltar que si es un factor a tomar en cuenta para que este proyecto se escale a almacenar millones de datos. Por el momento se dejó listas por el fácil manejo de datos y generar copias directas. No está de más mencionar que esto también se hizo para evitar el uso de la librería vector. También por ese factor de lista enlazada fue que se tomó la decisión de poner como claves 5. Pero como tal en ningún momento afecta a las pruebas el árbol sigue demostrando superioridad total en todo lo que hace. Llegando a ser hasta 100 veces más rápido que las listas enlazadas.

Análisis de complejidad Árbol B+

Esta es la tercera estructura de datos y sin duda alguna la más potente que tiene el programa teniendo un árbol B+. Este árbol por decisión del programador se declaró de grado 5 claves y su parámetro de orden son las categorías. Por lo tanto se tenían:

- 5 claves y como máximo 9 antes del desborde
- Por lo tanto para la realización del proyecto se tenían listas de máximo 10 hijos. Esto con el fin de poder manejar el árbol de la manera correcta respetando las 9 claves disponibles.

Este es uno de los árboles más increíbles ya que gracias al nivel que se le dio apenas si escala verticalmente. Esta decisión fue tomada debido a que los volúmenes de datos esperados para este proyecto y brindar escalabilidad horizontal en relación a que escale verticalmente de forma más amigable. Se calcula que para 10000 datos aproximadamente se tendría una altura de:

$$\log(1000) = 3 \text{ niveles}$$

Búsqueda: Este árbol se basa en comparar en su lista de claves que tiene 2d -1 claves y teniendo 2d nodos esto para poder generar las conexiones en base al movimiento de los índices por ciclos y también es importante mencionar que únicamente hará 3 saltos para poder llegar hasta las hojas. buscando el intervalo en el que hacen match todas las categorías que se suponen que están por el camino ya trazado por las hojas. Ya que este hace un descenso completamente vertical hasta las hojas. Para este árbol se tiene una complejidad de búsqueda del:

$$\mathbf{O}(log_m(\mathbf{n}))$$

Inserción: Esta va de la mano también con la búsqueda sin embargo tiene que ir verificando recursivamente si se desborda la lista de claves se propaga la separación de la lista hacia la raíz para poder ir verificando de tal forma de saber si se desencadena la partición de las listas. Y a diferencia del árbol B este solo sube las referencias de tal forma que se pueden generar caminos para llegar más rápido a las hojas. Sin embargo esto sigue siendo con la misma complejidad linealitmica:

$$\mathbf{O}(log_m(\mathbf{n}))$$

Nota: El parámetro de inserción debido a que pueden haber categorías iguales y es demasiado frecuente que estas se repitan y se concatenan con la llave primaria que era el código de barras con la fecha para poder seguir ordenando alfabéticamente.

Inserción: Esta va de la mano también con la búsqueda sin embargo tiene que ir verificando recursivamente si se desborda la lista de claves se propaga el balanceo mediante los préstamos y fusiones que pueden existir de la lista de claves hacia la raíz para poder ir verificando de tal forma de saber si se desencadenan fusiones o préstamos, Pero este no elimina las referencias solo borra la hoja pero la referencia la mantiene hacia el nodo que conecta a las siguientes listas es decir que es un proceso más simplificado ya que casi no pasan tanto los merges son más préstamos y de ultimo están las fusiones. Su complejidad se mantiene en:

$$\mathbf{O}(log_m(\mathbf{n}))$$

Aterrizaje vertical: Es el tiempo que se tarda en bajar desde la raíz hasta la primera hoja. En este caso al ser de grado 5 este permite generar menos rutas para llegar hasta las raíces. y de esta forma permite generar una búsqueda extremadamente rápida. Para el caso de los 10000 productos son únicamente 3 saltos y se descartan miles de datos en segundos. Una vez se llega hasta la primera hoja dentro del intervalo buscado. se empieza a recorrer horizontalmente recolectando datos. Y es sumamente eficiente por lo tanto en su fórmula K son los datos que le siguen al ir listando mediante el enlace que tienen todas las hojas hacia enfrente.

$$\mathbf{O}(log_m(\mathbf{n} + \mathbf{k}))$$

Cálculos de tiempos empíricos:

En este proyecto fue muy importante cronometrar cuánto tiempo le tomaba al árbol B+ buscar datos. En este caso está siempre insertaba con la garantía que siempre estaría ordenado por el parámetro de orden del árbol que era la categoría, debido a que está árbol desciende de forma vertical hasta las hojas y empieza a buscar horizontalmente. El promedio de comparaciones respecto a los 1000 productos evaluados, este llega extremadamente rápido hasta las hojas y como pruebas:

$$log_{10}\textbf{(n) -> } log_{10}\textbf{(1000) = 3 saltos}$$

Por lo tanto se puede conjeturar que el costo de búsquedas es muy rapida por el descenso que le toma muy pocos niveles al ser de grado 5.

Ejemplo empírico: (Casos muy extremos)

- Búsquedas:
    - Por categoría: 0.080 - 0.109 ms
- 5 veces

Promedio:

- Tiempo de búsqueda vertical: 0.020 ms
- Recorrido horizontal: 0.74 ms
- Tiempo promedio por categoría: 0.0945 ms aproximadamente en adelante

Justificación: El uso de listas enlazadas para poder crear el árbol B fue tomada debido a que en su debido momento se pensó a una escala de miles de datos. Sin embargo es importante tomar en cuenta que esto puede afectar al rendimiento si se colocan millones de datos, pero esto también tiene un trasfondo y es que generar la copia de arreglos al momento en el que se desbordan las claves está causaba problemas al perder referencias sin embargo es importante resaltar que si es un factor a tomar en cuenta para que este proyecto se escale a almacenar millones de datos. Por el momento se dejó listas por el fácil manejo de datos y generar copias directas. No está de más mencionar que esto también se hizo para evitar el uso de la librería vector. También otro motivo del porqué poner listas enlazadas fue por el factor de claves que fueron 10 y realmente no afecta demasiado al rendimiento ya que puede seguir ganando en comparación a todas las demás estructuras. Sin embargo **no está demás destacar que este árbol está diseñado para poder manejar grandes cantidades de datos. Es decir que realmente en pruebas de estrés se está sesgando porque no tiene suficientes datos para poder ejercer la diferencia. y en pruebas de búsquedas enormes. Realmente si gana siendo hasta 25 veces mejor que todas las demás estructuras implementadas ya no solo las listas sino que también se lleva por delante a los demás árboles.**

Metodología de medición utilizada

Se utilizaron las propias fórmulas de la definición de la notación Big-O para poder determinar aproximadamente cuántos caminos o recorridos tenía que tomar cada estructura de datos para poder llegar al resultado esperado. Al igual que se realizaron cálculos correspondientes en los promedios estimados. Sin embargo el núcleo de medición siempre fue el proyecto ya que este no cuenta con tanta latencia debido al framework utilizado que permite usar hilos y manejar concurrencia de forma muy sencilla. Por lo tanto el delay entre la medición y el tiempo real es de apenas 0.000001 nano segundos. Cosa que es muy acertada para los tiempos de búsqueda de cada estructura.

Como tal siempre se optó por una calculadora para sacar promedios en base a los resultados obtenidos por las búsquedas. Con más de 1000 datos.

Resultados

Se le presentan las tablas utilizadas para poder contabilizar los tiempos y poder calcular los promedios en base a la cantidad de pruebas hechas con más de 1000 productos almacenados dentro de cada estructura de datos.

Lista enlazada no ordenada:

Tiempo	Parámetro	Promedio
2.979	Nombre	3.20
1.708	Nombre	
1.668	Nombre	
3.012	Nombre	
2.965	Nombre	

Tiempo	Parámetro	Promedio
1.717	Categoría	3.2
1.708	Categoría	
1.982	Categoría	
3.760	Categoría	
2.743	Categoría	

Tiempo	Parámetro	Promedio
1.947	Fecha	2.5
1.923	Fecha	
1.982	Fecha	
1.802	Fecha	
1.905	Fecha	

Lista enlazada ordenada:

Tiempo	Parámetro	Promedio
2.583	Nombre	2.944
3.150	Nombre	
3.802	Nombre	
2.665	Nombre	
2.522	Nombre	

Tiempo	Parámetro	Promedio
1.717	Categoría	1.982
1.708	Categoría	
1.982	Categoría	
1.760	Categoría	
2.743	Categoría	

Tiempo	Parámetro	Promedio
2.161	Fecha	2.5
2.252	Fecha	
2.218	Fecha	
2.01	Fecha	
2.80	Fecha	

Arbol AVL:

Tiempo	Parámetro	Promedio
0.010	Nombre	0.0374
0.15	Nombre	
0.009	Nombre	
0.010	Nombre	
0.008	Nombre	

Árbol B:

Tiempo	Parámetro	Promedio
0.098	Fecha	0.988
0.097	Fecha	
0.111	Fecha	
0.063	Fecha	
0.125	Fecha	

Arbol B+:

Tiempo	Parámetro	Promedio
0.172	Categoría	0.161
0.132	Categoría	
0.170	Categoría	
0.193	Categoría	
0.138	Categoría	

Conclusiones

Los árboles son las estructuras más potentes para poder almacenar datos por la eficiencia que ofrecen y la lógica que estos llevan pese a ser compleja media vez se entiende el concepto se puede observar que son demasiado eficientes las búsquedas con estos árboles. Y que en comparación a listas enlazadas les sacan mucha ventaja. Por lo tanto son las mejores alternativas para hacer un proyecto formal que permita almacenar datos dentro de sí mismo y nos demuestra que una elección correcta de estructuras de datos nos puede generar una eficiencia increíble en las latencias de búsqueda que pueden existir dentro de cualquier programa.

El árbol B+ es la estructura de datos más rápida y eficiente que existe por su búsqueda extremadamente rápida que ofrece al tener referencias en el descenso vertical que éste hace para poder proceder a buscar grandes cantidades de datos. Sin siquiera recorrer recursivamente y subir de niveles. Es una búsqueda binaria bastante rápida ya que todos sus datos están en las hojas y solo tiene puentes para determinar dónde empezar su búsqueda y hasta donde llegar.

Las listas enlazadas solo son implementaciones puntuales. Estas realmente no son recomendables para las búsquedas de datos o manejo de millones de datos ya que tiene una búsqueda secuencial y llegando a dar como resultado complejidades de código demasiado grandes por esta misma característica que tiene de recorrer nodo por nodo la lista hasta llegar a los resultados buscados.